

DOKUMENTASI PENGGUNAAN AI CODING TOOL

1. IDENTITAS DAN INFORMASI PROYEK

Nama Proyek	Sistem Perpustakaan Berbasis Web
Jenis Pengembangan	Pengembangan menjadi 2 service / microservice
Service 1	Book Service
Service 2	Borrowing Service
AI Coding Tool	Gemini 3.1 & ChatGPT-5.6 Luna

2. TAHAPAN PENGGUNAAN AI CODING TOOL

No.	Tahap	Tujuan	Prompt yang Digunakan	Hasil/Tindakan
1	Review dan perencanaan Arsitektur	Mengevaluasi proyek perpustakaan yang sudah ada dan menentukan pembagian tanggung jawab service	Saya sedang melanjutkan proyek sistem peminjaman buku perpustakaan yang sudah saya kembangkan sebagian menjadi arsitektur microservice. Saya butuh bantuan kamu sebagai AI coding Tool untuk mereview, memperbaiki dan melengkapi kode yang sudah ada – bukan membuat dari nol	AI memberikan panduan Langkah-langkah pengerjaan, memberikan Struktur folder proyek microservice. Kemudian mulsi pengerjaan yakni tahap pertama AI memberikan service 1: Book Service. Service 2: Borrowing Service
2	Pembuatan Service Buku	Service ini khusus untuk	Ubah sistem peminjaman buku	Menampilkan semua data

		mengelola katalog buku dan status ketersediaan buku.	yang sebelumnya menggunakan JavaScript localStorage menjadi arsitektur microservice. Buat Book Service menggunakan Node.js dan Express untuk mengelola data buku dan status ketersediaan buku.	buku sebelumnya kemudian, membuat serve/API service book.
3	Pembuatan Service Peminjaman	Membuat server/API Service Peminjaman	Buat Borrowing Service menggunakan Node.js dan Express. Service harus memvalidasi maksimal 3 buku aktif per mahasiswa dan masa peminjaman 7 hari.”	Kode diperiksa dan disesuaikan dengan kebutuhan proyek.
4	Komunikasi API	Menghubungkan kedua service	Implementasikan komunikasi API antara Borrowing Service dan Book Service untuk mengecek ketersediaan buku sebelum transaksi peminjaman.	Endpoint dan alur komunikasi diuji.
5	Debugging	Memperbaiki error	Kode saya menghasilkan error berikut: [tempel error].	Solusi AI diuji dan hanya perbaikan yang sesuai yang

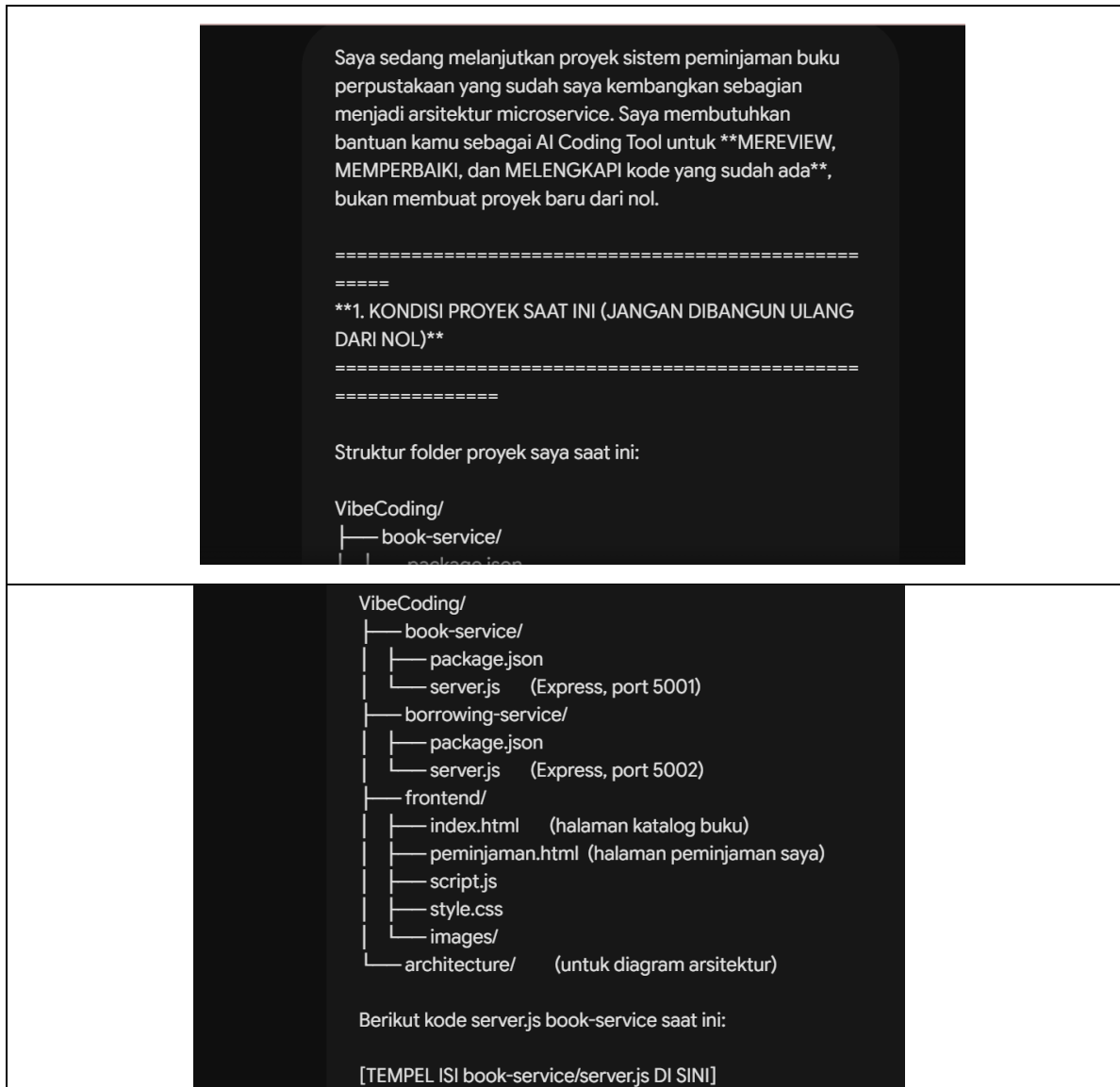
			Apa penyebab dan cara memperbaikinya?	diterapkan.
--	--	--	---------------------------------------	-------------

No	Bagian Yang Diperiksa	Masalah/Hasil Pemeriksaan	Tindakan Perbaikan
1.	Port Server	URL dan Port <i>borrowing-service</i> dipanggil langsung secara <i>hardcode</i> (http://localhost:5001), serta belum ada penanganan port dinamis dari <i>environment variable</i> .	Mengubah konfigurasi port agar menggunakan <code>process.env.PORT</code> dengan <i>fallback</i> port default (Book Service: 5001 , Borrowing Service: 5002 , dan Auth Service: 5003). Menjadikan URL komunikasi dinamis melalui <code>process.env.BOOK_SERVICE_URL</code> .
2.	Endpoint API	1. Belum ada endpoint untuk mekanisme login/autentikasi mahasiswa (BR-01 / AC-01). 2. Pengecekan ketersediaan buku dan pembaruan status buku dilakukan secara terpisah, berisiko <i>race condition</i> (AC-03). 3. Belum ada endpoint pengembalian buku (<i>return</i>).	1. Membuat service otentikasi baru (<i>auth-service</i>) dengan endpoint POST <code>/api/auth/login</code> dan POST <code>/api/auth/verify</code>. 2. Membuat endpoint atomic PUT <code>/api/books/:id/borrow</code> pada <i>book-service</i> untuk langsung mengecek dan mengunci status buku. 3. Menambahkan endpoint POST <code>/api/borrowings/:id/return</code> dan PUT <code>/api/books/:id/return</code> untuk fitur pengembalian buku.
3.	Komunikasi antar-service	1. <i>Borrowing-service</i> menerima <i>nim</i> langsung dari <i>request body</i> tanpa memvalidasi identitas	1. Menambahkan middleware <code>authenticateToken</code> pada <i>borrowing-service</i> untuk memverifikasi token ke <i>auth-service</i> via HTTP REST API

		ke service mana pun. 2. Tidak ada <i>error handling</i> jika <i>book-service</i> sedang mati/tidak merespons.	sebelum memproses peminjaman. 2. Menambahkan blok try-catch dan penanganan status HTTP 503 Service Unavailable saat komunikasi antar-service gagal.
--	--	--	--

3. BUKTI SCREENSHOT PENGGUNAAN AI

Screenshot 1. Percakapan AI saat merancang arsitektur 2 service.



Berikut kode server.js borrowing-service saat ini:

[TEMPEL ISI borrowing-service/server.js DI SINI]

Berikut kode frontend saya saat ini:

[TEMPEL ISI index.html, peminjaman.html, dan script.js DI SINI — atau upload file langsung ke AI Coding Tool]

****Catatan tentang kondisi saat ini:****

- * book-service menyimpan 10 data buku di memori menggunakan array JavaScript, bukan database.
- * book-service memiliki endpoint GET `/api/books`, GET `/api/books/:id`, dan PUT `/api/books/:id/status`.
- * borrowing-service menyimpan data peminjaman di memori menggunakan array.
- * borrowing-service memiliki endpoint GET `/api/borrowings/:nim` dan POST `/api/borrowings`.

- * borrowing-service memiliki endpoint GET `/api/borrowings/:nim` dan POST `/api/borrowings`.
- * Saat proses peminjaman, borrowing-service memanggil book-service melalui HTTP untuk mengecek ketersediaan buku dan mengubah status buku menjadi tidak tersedia.
- * Validasi maksimal 3 buku aktif sudah ada di borrowing-service.
- * Mekanisme login atau autentikasi mahasiswa belum tersedia secara nyata pada backend. Field `'nim'` pada request peminjaman masih diterima langsung dari request dan belum diverifikasi melalui mekanisme autentikasi.
- * **Fitur pengembalian buku sudah tersedia pada sistem sebelumnya, sehingga fitur tersebut tidak boleh dianggap sebagai fitur baru. Namun, implementasi pengembalian pada sisi backend/microservice perlu direview dan dikembangkan agar dapat terintegrasi dengan Borrowing Service dan Book Service.**
- * URL antar-service saat ini masih menggunakan URL yang ditulis langsung di dalam kode dan perlu direview agar dapat

	<pre> ===== **2. USER STORY (WAJIB DIPERTAHANKAN, TIDAK BOLEH DIUBAH)** ===== ===== * US-01: Sebagai mahasiswa, saya ingin mendapatkan peringatan jika meminjam buku lebih dari 7 hari, sehingga saya tidak lupa mengembalikan buku tersebut. * US-02: Sebagai mahasiswa, saya ingin melihat daftar buku beserta status ketersediaannya, sehingga saya dapat mengetahui buku yang masih bisa dipinjam. * US-03: Sebagai mahasiswa, saya ingin mendapatkan peringatan ketika jumlah buku yang saya pinjam sudah mencapai batas maksimal, sehingga saya mengetahui bahwa saya tidak dapat meminjam buku lagi. ===== </pre>	
	<pre> ===== **3. ACCEPTANCE CRITERIA (WAJIB DIPERTAHANKAN, TIDAK BOLEH DIUBAH)** ===== ===== * AC-01: Given mahasiswa sudah login dan buku tersedia, When mahasiswa memilih tombol pinjam, Then sistem berhasil memproses peminjaman buku. * AC-02: Given mahasiswa sudah memiliki 3 buku aktif, When mahasiswa mencoba meminjam buku ke-4, Then sistem menolak peminjaman. * AC-03: Given buku sedang dipinjam oleh mahasiswa lain, When mahasiswa mencoba meminjam buku tersebut, Then sistem menolak peminjaman. * AC-04: Given peminjaman berhasil dilakukan, When mahasiswa melihat informasi peminjaman, Then sistem menampilkan tanggal peminjaman dan tanggal jatuh tempo 7 </pre>	
	<pre> **Catatan penting:** AC-01 mensyaratkan mahasiswa sudah login. Kondisi ini belum terpenuhi sepenuhnya pada implementasi backend saat ini karena belum terdapat mekanisme autentikasi nyata. Hal tersebut perlu diperbaiki tanpa mengubah acceptance criteria yang sudah ditetapkan. ===== **4. BUSINESS RULES (WAJIB DIPERTAHANKAN, TIDAK BOLEH DIUBAH)** ===== ===== 1. Mahasiswa wajib login sebelum dapat melakukan peminjaman buku. 2. Hanya buku yang berstatus tersedia yang dapat dipinjam. 3. Satu buku hanya dapat dipinjam oleh satu mahasiswa pada </pre>	

	1. Mahasiswa wajib login sebelum dapat melakukan peminjaman buku. 2. Hanya buku yang berstatus tersedia yang dapat dipinjam. 3. Satu buku hanya dapat dipinjam oleh satu mahasiswa pada satu waktu. 4. Setiap mahasiswa maksimal memiliki 3 buku aktif yang sedang dipinjam. 5. Masa peminjaman adalah 7 hari sejak tanggal peminjaman. 6. Sistem harus mencatat dan menampilkan informasi peminjaman, yaitu buku yang dipinjam, tanggal peminjaman dan tanggal jatuh tempo. 7. Setelah buku dipinjam, status buku berubah menjadi tidak tersedia sehingga mahasiswa lain tidak bisa meminjam buku yang sama. Selain itu, proses pengembalian buku yang sudah tersedia sebelumnya harus tetap dipertahankan. Ketika buku dikembalikan, sistem harus memastikan data peminjaman diperbarui dan status buku pada Book Service kembali	
	Selain itu, proses pengembalian buku yang sudah tersedia sebelumnya harus tetap dipertahankan. Ketika buku dikembalikan, sistem harus memastikan data peminjaman diperbarui dan status buku pada Book Service kembali menjadi tersedia. ===== ===== **5. TUJUAN TUGAS (INI YANG HARUS DIPENUHI DARI KODE YANG SUDAH ADA)** ===== ===== Instruksi resmi dari dosen untuk tugas ini: a. Gunakan proyek yang telah dikerjakan sebelumnya sebagai basis dan jangan membangun ulang proyek dari nol. b. Gunakan kembali User Story dan Acceptance Criteria yang	
	5. TUJUAN TUGAS (INI YANG HARUS DIPENUHI DARI KODE YANG SUDAH ADA) ===== ===== Instruksi resmi dari dosen untuk tugas ini: a. Gunakan proyek yang telah dikerjakan sebelumnya sebagai basis dan jangan membangun ulang proyek dari nol. b. Gunakan kembali User Story dan Acceptance Criteria yang sudah ada. c. Kembangkan arsitektur proyek menjadi minimal 2 service/microservice. Saat ini sudah terdapat book-service dan borrowing-service, tetapi perlu dievaluasi apakah pembagian tanggung jawabnya sudah tepat dan lengkap, terutama terkait autentikasi mahasiswa.	

=====
7. CATATAN PENTING
=====

- * Jangan menghapus atau mengubah 10 data buku yang sudah ada di book-service kecuali benar-benar diperlukan secara teknis.
- * Jangan membangun ulang proyek dari nol.
- * Jangan mengubah User Story dan Acceptance Criteria.
- * Jangan menghilangkan fitur yang sudah tersedia.
- * **Fitur pengembalian buku bukan fitur baru; fitur tersebut sudah tersedia sebelumnya dan harus dipertahankan serta dikembangkan implementasinya pada arsitektur microservice.**
- * Jangan menambahkan fitur di luar yang diminta, seperti fitur admin, denda, atau notifikasi, kecuali memang diperlukan untuk memenuhi kebutuhan yang sudah disebutkan.
- * Prioritaskan kode yang benar secara fungsional dan mudah

Panduan Langkah-Langkah Pengerjaan

1. Ubah Arsitektur menjadi Microservice (Minimal 2 Service)

- o **Sebelum:** Monolith / Single Page Static (HTML + CSS + JS Vanilla + localStorage).
[PDF](#)
- o **Sesudah:** Pisahkan sistem menjadi minimal 2 service independen yang memiliki tugas spesifik.
- o **Contoh Pembagian:**
 - o **Auth/User Service:** Menangani login mahasiswa & data akun. [PDF](#)
 - o **Book/Catalog Service:** Menangani daftar buku & status ketersediaan. [PDF](#)
 - o **Borrowing Service:** Menangani transaksi peminjaman, aturan batas 3 buku, & tanggal jatuh tempo 7 hari. [PDF](#)
 - o (Pilih minimal 2 service, misalnya: **Catalog Service** dan **Borrowing Service**).

2. Komunikasi Antar-Service via API

Struktur Folder Proyek Microservice

Plaintext

```
perpustakaan-microservice/  
├── frontend/  
│   ├── index.html  
│   ├── style.css  
│   └── script.js  
├── book-service/  
│   ├── package.json  
│   └── server.js  
├── borrowing-service/  
│   ├── package.json  
│   └── server.js  
├── architecture/  
│   └── architecture.png  
└── README.md
```

Screenshot 2. Percakapan AI saat membuat Book service dan Borrowing Service.

"Ubah sistem peminjaman buku yang sebelumnya menggunakan JavaScript localStorage menjadi arsitektur microservice. Buat Book Service menggunakan Node.js dan Express untuk mengelola data buku dan status ketersediaan buku."

1. Isi Kode book-service

- Masuk ke folder `book-service`, lalu buat file baru bernama `server.js`. JS+1
- Paste kode berikut ke dalamnya:

JavaScript

```
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());

let books = [
  { id: "BK01", judul: "Coding Basics", pengarang: "Andrea Hirata", gambar: "images/gambar1.jpg" },
  { id: "BK02", judul: "Computer Science", pengarang: "Pramoedya Ananta Toer", gambar: "images/gambar2.jpg" },
  { id: "BK03", judul: "Linux Kernal", pengarang: "Thierry Gayet", gambar: "images/gambar3.jpg" },
  { id: "BK04", judul: "Code Firts", pengarang: "Oreilly", gambar: "images/gambar4.jpg" }
];
```

JavaScript

```
// GET: Ambil semua buku
app.get('/api/books', (req, res) => {
  res.json(books);
});

// GET: Cek detail & ketersediaan buku tertentu
app.get('/api/books/:id', (req, res) => {
  const book = books.find(b => b.id === req.params.id);
  if (!book) return res.status(404).json({ message: "Buku tidak ditemukan" });
  res.json(book);
});

// PUT: Update status ketersediaan buku (dipanggil oleh Borrowing Service)
app.put('/api/books/:id/status', (req, res) => {
  const { isAvailable } = req.body;
  const book = books.find(b => b.id === req.params.id);
  if (!book) return res.status(404).json({ message: "Buku tidak ditemukan" });

  book.isAvailable = isAvailable;
  res.json({ message: "Status buku berhasil diperbarui", book });
});
```

2. Service 2: Borrowing Service (borrowing-service/server.js)

Service ini mengurus aturan bisnis peminjaman (maksimal 3 buku, jatuh tempo 7 hari) dan berkomunikasi dengan Book Service menggunakan `fetch` / `axios`.

JavaScript

```
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());
app.use(express.json());

let loans = [];

// Helper: Hitung tanggal jatuh tempo 7 hari
```

JavaScript

```
// Helper: Hitung tanggal jatuh tempo 7 hari
function calculateDueDate() {
  const date = new Date();
  date.setDate(date.getDate() + 7);
  return date.toISOString().split('T')[0];
}

// GET: Ambil peminjaman berdasarkan NIM
app.get('/api/borrowings/:nim', (req, res) => {
  const userLoans = loans.filter(l => l.nim === req.params.nim && l.status === 'AKTIF');
  res.json(userLoans);
});

// POST: Proses Transaksi Peminjaman (Melibatkan Komunikasi API ke Book Service)
app.post('/api/borrowings', async (req, res) => {
  const { nim, bookId } = req.body;

  // 1. Validasi Aturan Bisnis: Batas Maksimal 3 Buku
  const activeLoans = loans.filter(l => l.nim === nim && l.status === 'AKTIF');
  if (activeLoans.length >= 3) {
```

```
const bookResponse = await fetch(`http://localhost:5001/api/books/${bookId}`);
if (!bookResponse.ok) {
  return res.status(404).json({ message: "Buku tidak ditemukan pada katalog" });
}
const book = await bookResponse.json();

// Validasi Ketersediaan Buku
if (!book.isAvailable) {
  return res.status(400).json({ message: "Buku sedang tidak tersedia untuk dipinjam" });
}

// 3. Buat Transaksi Peminjaman Baru
const newLoan = {
  id: `LN-${nim}-${Date.now()}`,
  nim,
  bookId,
  tanggalPinjam: new Date().toISOString().split('T')[0],
  tanggalJatuhTempo: calculateDueDate(),
  status: 'AKTIF'
};
```

```
};
loans.push(newLoan);

// 4. Update Status Buku di Book Service menjadi Tidak Tersedia
await fetch('http://localhost:5001/api/books/${bookId}/status', {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ isAvailable: false })
});

res.status(201).json({ message: "Peminjaman berhasil", loan: newLoan });
} catch (error) {
  res.status(500).json({ message: "Gagal terhubung ke Book Service", error: error });
}
};

p.listen(5002, () => console.log('Borrowing Service berjalan di port 5002'));
```

Screenshot 3. Percakapan AI saat membuat/menjelaskan komunikasi API.

1. Update File `frontend/script.js`

Buka file `frontend/script.js` kamu, lalu sesuaikan bagian yang bertugas mengambil data buku dan melakukan peminjaman agar memanggil API backend:

- **Untuk Mengambil Data Buku (Ganti `fetch`/data lokal lama dengan API Book Service):**

JavaScript

```
// Ambil data buku dari Book Service (Port 5001)
async function fetchBooks() {
  try {
    const response = await fetch('http://localhost:5001/api/books');
    const books = await response.json();
    renderBooks(books); // Panggil fungsi render UI kamu
  } catch (error) {
    console.error('Gagal mengambil data buku:', error);
  }
}
```

+ Minta Gemini

Flash

1. Komunikasi Frontend ke `book-service` (Port 5001)

- o **Alur API:** `frontend/script.js` mengirim permintaan `GET` ke `http://localhost:5001/api/books` untuk mengambil daftar katalog buku.
- o **Error:** Muncul pesan kesalahan *CORS Policy* di *console browser* (`Failed to fetch`), sehingga daftar buku tidak bisa tampil.
- o **Perbaikan:** Menambahkan *middleware* `CORS` di `book-service/server.js`:

JavaScript

```
const cors = require('cors');  
app.use(cors());
```

2. Komunikasi Frontend ke `borrowing-service` (Port 5002)

+ Minta Gemini

Flash



Gemini adalah AI dan dapat membuat kesalahan.

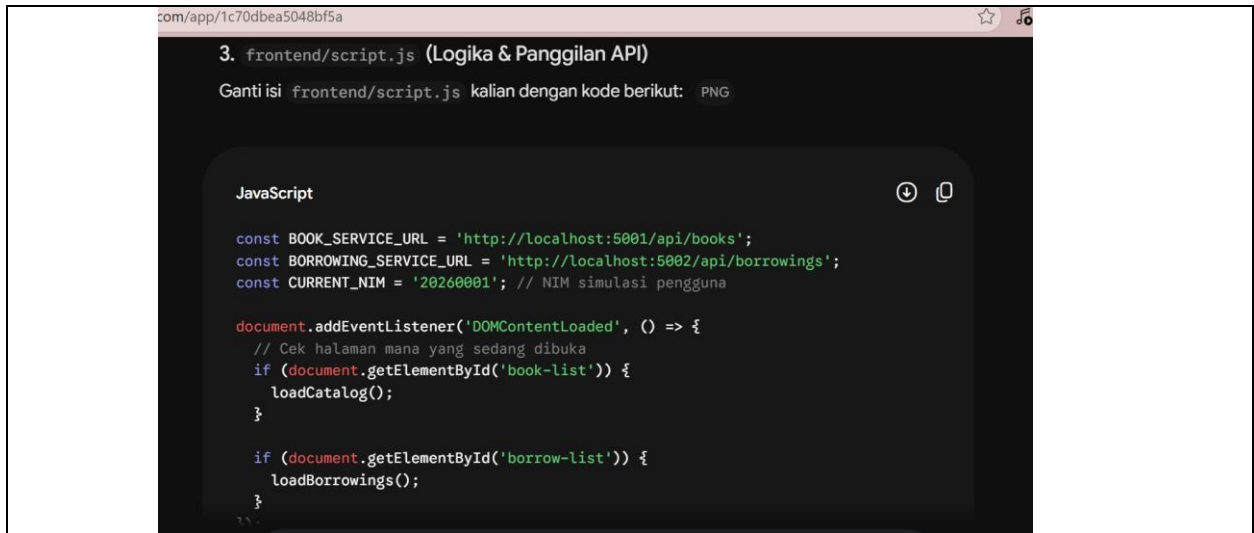
2. Komunikasi Frontend ke `borrowing-service` (Port 5002)

- o **Alur API:** Saat tombol "Pinjam" diklik, `frontend/script.js` mengirim permintaan `POST` ke `http://localhost:5002/api/borrowings`.
- o **Error:** Permintaan gagal/berbentrok (*Port Conflict*) karena AI sempat mengatur `borrowing-service` berjalan di port yang sama dengan `book-service` (5001).
- o **Perbaikan:** Memisahkan port di `borrowing-service/server.js` menjadi port 5002 :

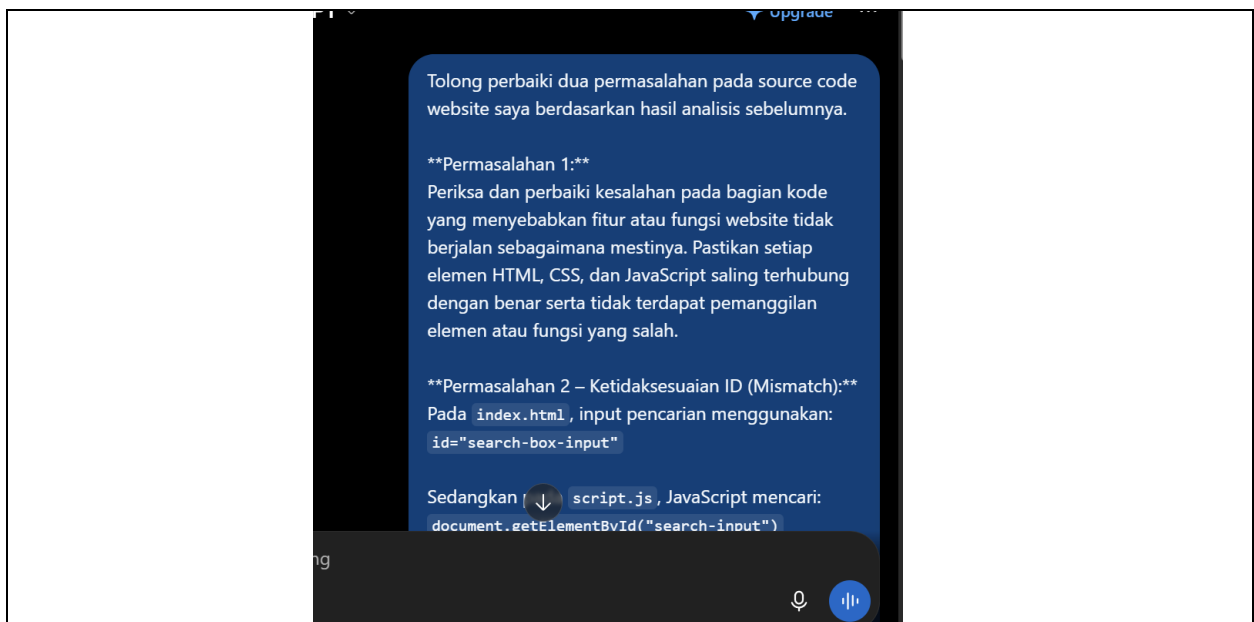
JavaScript

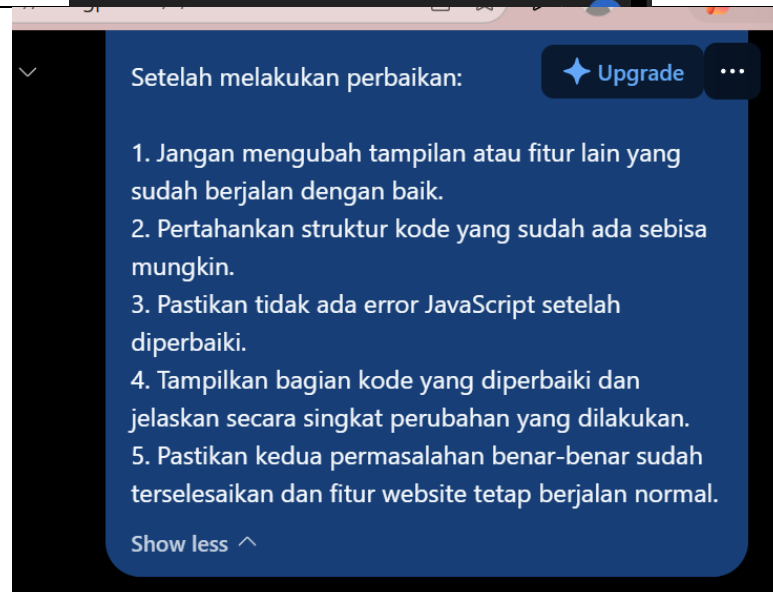
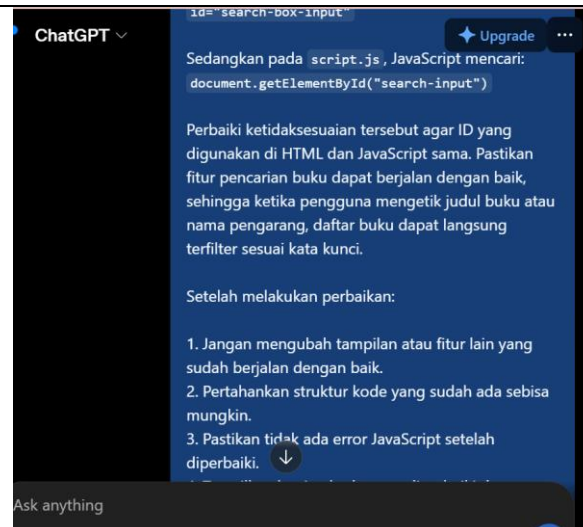
```
const PORT = 5002;  
app.listen(PORT, () => console.log(`Borrowing Service running on port ${PORT}`))
```

3. Komunikasi Antar-Service (Inter-Service API Communication)



Screenshot 4. Percakapan AI saat membantu memperbaiki error.





o Untuk Peminjaman Buku (Ganti logika pinjam lama dengan API Borrowing Service):

JavaScript

```
// Kirim transaksi peminjaman ke Borrowing Service (Port 5002)
async function handleBorrow(bookId) {
  const nim = "20260001"; // Sesuaikan dengan NIM/User yang sedang login

  try {
    const response = await fetch('http://localhost:5002/api/borrowings', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ nim, bookId })
    });

    const result = await response.json();

    if (!response.ok) {
      alert(result.message); // Menampilkan pesan misal sudah capai batas 3 buku
      return;
    }
  }
}
```

Susunan letak folder dan file-filenya adalah sebagai berikut:

Plaintext

```
perpustakaan-microservice/ <-- FOLDER UTAMA PROYEK
├── frontend/ <-- FOLDER FRONTEND
│   ├── index.html (file index.html lama)
│   ├── style.css (file style.css lama)
│   └── script.js (file script.js yang sudah diubah dipanggil API)
├── book-service/ <-- FOLDER BACKEND 1
│   ├── package.json (dibuat pakai command: npm init -y)
│   └── server.js (isi dengan kode Book Service)
├── borrowing-service/ <-- FOLDER BACKEND 2
│   ├── package.json (dibuat pakai command: npm init -y)
│   └── server.js (isi dengan kode Borrowing Service)
```

```

Plaintext
├── package.json      (about package manager, npm, etc) //
├── server.js         (isi dengan kode Borrowing Service)
├── architecture/     <-- FOLDER DOKUMENTASI
│   └── architecture.png (gambar diagram microservice)
└── README.md         <-- FILE PENJELASAN PROYEK

```

Cara Menjalankan & Membuka di VS Code

Buat Folder Utama

Buat folder baru di komputer dengan nama `perpustakaan-microservice`.

Buka di VS Code

Buka VS Code, lalu pilih **File > Open Folder** dan pilih folder `perpustakaan-microservice` tersebut.

3. Buat Sub-folder & File

Buat folder `frontend`, `book-service`, `borrowing-service`, dan `architecture` di dalam VS Code sesuai struktur di atas.

4. Inisialisasi Node.js di Masing-Masing Service

Buka Terminal di VS Code, lalu jalankan perintah berikut secara berurutan:

o Untuk Book Service:

```

Bash
cd book-service
npm init -y
npm install express cors

```

o Untuk Borrowing Service:

```

Bash
cd ../borrowing-service
npm init -y
npm install express cors

```

5. Cara Menjalankan Sistem Saat Testing:

Buka 2 Terminal di VS Code:

- o **Terminal 1:** Masuk ke folder `book-service` lalu jalankan `node server.js` (berjalan di port 5001).
- o **Terminal 2:** Masuk ke folder `borrowing-service` lalu jalankan `node server.js` (berjalan di port 5002).
- o **Frontend:** Buka file `frontend/index.html` langsung di browser atau gunakan ekstensi *Live Server* di VS Code.

Pada perbaikan kami juga menggunakan bantuan chat gpt.

4. HASIL IMPLEMENTASI

Setelah dilakukan pengembangan, sistem perpustakaan yang sebelumnya berada dalam satu folder telah dipisahkan menjadi dua service, yaitu:

1. Book Service : Digunakan untuk mengelola data buku dan mengecek apakah buku tersedia atau sedang dipinjam.
2. Borrowing Service : Digunakan untuk mengelola proses peminjaman dan pengembalian buku.

Kedua service dijalankan pada server dan port yang berbeda. Book Service menggunakan port 5001, sedangkan Borrowing Service menggunakan port **5002**. Kedua service dapat saling berkomunikasi menggunakan **API**. Contohnya, ketika pengguna ingin meminjam buku, Borrowing Service akan meminta informasi kepada Book Service untuk mengecek apakah buku tersebut masih tersedia. Selain itu, sistem juga memiliki aturan peminjaman, yaitu mahasiswa dapat meminjam maksimal **3 buku** dan lama peminjaman adalah **7 hari**. Hasil pengembangan kemudian diuji dengan menjalankan kedua server dan mencoba proses peminjaman serta pengembalian buku. Dari pengujian tersebut, komunikasi antar-service dan alur peminjaman dapat dijalankan sesuai dengan fungsi yang telah dibuat.

5. KESIMPULAN

Penggunaan AI Coding Tool membantu proses pengembangan proyek perpustakaan menjadi arsitektur dua service. AI digunakan untuk membantu perancangan, pembuatan kode, pemahaman API, serta pemecahan masalah. Meskipun demikian, hasil dari AI tetap diperiksa dan disesuaikan oleh pengembang agar sesuai dengan kebutuhan proyek dan dapat berjalan dengan baik.